Project Executable Files

SMARTBRIDGE - Project Executable Files

- Date: 15/10/2026
- Team ID: SWTID-2026-4348
- Team Size: 1
- Team Leader:Naveen
- Project Name: AI FitTrack
- **Maximum Marks:** 3 Marks
-

Maximum Marks: 3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)	Remarks / Details
1	Complete source code (all files and folders)	Yes	Full Node.js/Express.js backend source code included.
2	README / Setup Guide (instructions to run)	Yes	Complete README.md file located in the root directory.
3	requirements.txt / package.json / dependency file	Yes	package.json & package-lock.json included.
4	Database schema / seed files (if applicable)	Yes	Mongoose Schema files in src/models/User.js & src/models/Workout.js.
5	Environment configuration file (.env.example)	Yes	.env.example and configured .env included.
6	Deployed application URL / Executable binary	Yes	Local REST API server at http://localhost:5000.
7	Dockerfile / Containerization config (if applicable)	NA	Standard Node.js runtime environment used.
8	APK (if applicable)	NA	Backend RESTful API service.
9	Test files and test results	Yes	Postman Collection (FitTrack.postman_collection.json) & test logs.
10	Demo video or walkthrough	Yes	Google Drive walkthrough & API demo video links provided.

Step 2: File / Folder Structure

Code Files /
├── node_modules/
├── src/
│ ├── config/

```
│   │   └── db.js # MongoDB connection configuration using Mongoose ODM
│   ├── controllers/
│   │   ├── aiController.js # Handles Gemini AI recommendations and insights logic
│   │   ├── authController.js # User registration, JWT login, and profile controllers
│   │   └── workoutController.js # CRUD operations for workout management
│   ├── middleware/
│   │   ├── auth.js # Express JWT authentication & token verification
│   │   └── errorHandler.js # Centralized error response handling middleware
│   ├── models/
│   │   ├── User.js # Mongoose schema for User collection (with bcrypt)
│   │   └── Workout.js # Mongoose schema for Workout collection
│   ├── routes/
│   │   ├── aiRoutes.js # Express router endpoints for AI services
│   │   ├── authRoutes.js # Express router endpoints for authentication
│   │   ├── index.js # Central route registry
│   │   └── workoutRoutes.js # Express router endpoints for workout CRUD
│   ├── services/
│   │   └── geminiService.js # Google Gemini API (@google/genai) service handler
│   ├── app.js # Express application setup, CORS, and routing middleware
│   └── server.js # Server startup entry point and DB initialization
├── .env # Local environment variables (PORT, MONGO_URI, GEMINI_API_KEY)
├── .env.example # Environment variable blueprint
├── .gitignore # Git ignore file for node_modules and .env
├── FitTrack.postman_collection.json # API collection for Thunder Client / Postman testing
├── package-lock.json # Locked dependency versions
├── package.json # Project dependencies and npm startup scripts
└── README.md # Setup instructions, endpoints guide, and run commands
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	http://localhost:5000/api
Login Credentials (Demo)	Email: sathya@gmail.com Password: Sathya@01
Platform / Hosting Provider	Node.js Express Runtime & MongoDB Atlas Cloud Database
Repository Link	https://drive.google.com/drive/folders/1hwmKxkHQkUI62JR1svI3xCQ2ShZ6Lj9y

Demo Video Links

Demo Video:
https://drive.google.com/file/d/1KswrctSaPI0eOYyJ3GkbbVHvHsU3EfZJ/view?usp=drive_link

API Testing Video:
https://drive.google.com/file/d/1TCXG48a-78ryxF3gipdRMwRBwuhAgPaa/view?usp=drive_link

Step 4: Run Instructions

Pre-requisites: Node.js v16.0.0 or higher; npm v8.0.0 or higher; Active MongoDB Atlas Cloud Cluster or Local MongoDB; Google Gemini API Key (@google/genai).

Step 1: Open VS Code terminal and navigate into project directory: cd "Code Files"

Step 2: Install dependencies: npm install

Step 3: Configure environment variables in .env file: PORT=5000; MONGO_URI=<your MongoDB Atlas URI>; JWT_SECRET=<your JWT secret>; GEMINI_API_KEY=<your Gemini API key>; GEMINI_MODEL=gemini-2.5-flash

Step 4: Start the REST API application server: npm start

Step 5: Confirm running server status in console: Server running in development mode on port 5000; MongoDB Connected.

Step 6: Open Thunder Client or Postman and test endpoints at http://localhost:5000.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	ENOTFOUND MongoDB Connection Delay: Occurs if non-SRV connection strings are used or during internet drops.	Resolved: Upgraded to standard mongodb+srv:// URI format with Mongoose connection retry options.
2	Port Occupied (EADDRINUSE on Port 5000): Port 5000 remains occupied by an orphaned process.	Resolved: Executed npx kill-port 5000 before running npm start.
3	Gemini AI Deprecated Model Error (NOT_FOUND): Older model strings fail API calls.	Resolved: Updated model string to gemini-2.5-flash in .env and geminiService.js.